

EP 1 – Relatório

Ísis Ardisson Logullo
nUSP 7577410

Resultados dos Testes

Buscas	Tiny Maze		Medium Maze		Big Maze	
	Nós	Custo	Nós	Custo	Nós	Custo
DFS	15	10	146	130	390	210
BFS	15	8	269	68	620	210
UCS	15	8	269	68	620	210
A*	14	8	221	68	549	210
IDDFS	64	8	10063	68	60211	210

Busca	Medium Maze		Medium Dotted Maze (East)		Medium Scary Maze (West)	
	Nós	Custo	Nós	Custo	Nós	Custo
UCS	269	68	186	1	108	68719479864

Medium Maze	Trials = 10		Trials = 30		Trials = 50		Trials = 100		Trials = 150	
	Nós	Custo	Nós	Custo	Nós	Custo	Nós	Custo	Nós	Custo
LRTA	3658	76	7486	130	9950	110	15034	74	18874	68

O A* é um algoritmo de melhor desempenho em modo geral. Encontrou o caminho ótimo com menos expansões de nós, de 2 em 3 fases de maneira mais efetiva. O único caso em que ficou em segundo lugar foi no bigMaze, em que o DFS precisou de pouco mais da metade de expansões para achar um caminho ótimo. No entanto, visto que o DFS de maneira geral não é completo nem ótimo, o A* é uma escolha mais segura.

O BFS se saiu relativamente bem, fazendo por volta de 60 expansões a mais que o A*. Em problemas mais complexos, isso pode ser um problema. A principal vantagem é oferecer um caminho ótimo, assim como a UCS, que teve o mesmo número de expansões (provavelmente porque as ações tinham todas o mesmo custo). Mas isso muda completamente quando usamos os agentes com custo variado no UCS. O East tem um desempenho excelente, já o West é péssimo.

Já o IDDFS teve um número quase 100 vezes maior de expansões comparado ao BFS no bigMaze. A vantagem é a economia de memória, ele combina a economia de espaço da DFS com a otimalidade da BFS. Ele expande muitos nós repetidamente a cada aumento de profundidade, por isso o número de nós é tão alto, mas ele é ideal para situações onde a memória RAM é baixa.

O LRTA perde para a maioria e performa aceitavelmente com trials mais altos. Mas a quantidade de expansões são muito altas. Vai pior ainda que o IDDFS comparando apenas o medium. Por ser um algoritmo online, o LRTA aprende enquanto se move. O alto número de nós reflete o aprendizado acumulado ao longo das

várias tentativas. O custo final é igual ao custo do A*, então eventualmente ele aprende o caminho ótimo, mas ao custo de muita exploração.

1) Busca em Profundidade

A ordem de exploração seguiu o esperado. É possível ver na visualização da fase que o algoritmo tende a seguir uma "trilha" até o fim em vez de explorar todos os lados. Isso corresponde a seguir um caminho no grafo até atingir o estado meta ou encontrar uma folha, como esperado de uma busca em profundidade.

O tamanho da solução está de acordo. DFS não é um algoritmo ótimo. Explora um caminho até o fim antes de retroceder (*backtracking*). O tamanho da solução depende inteiramente de qual "ramo" do labirinto ela entra primeiro. Como ela segue cegamente a ordem da pilha, ela frequentemente encontra caminhos muito longos e tortuosos, ignorando atalhos que estariam em ramos ainda não explorados. Assim, as soluções *medium* e *big* tendem a ser maiores.

Se fosse uma Busca em Árvore, o algoritmo entraria em um loop infinito na maioria dos labirintos.

2) Busca em Largura

A implementação feita encontra uma solução de custo mínimo, pois explora todos os nós atingíveis a uma certa distância do estado inicial antes de explorar mais longe. Assim, obrigatoriamente encontra a primeira ocorrência possível do estado meta.

A BFS encontraria soluções diferentes apenas em termos de caminho. O custo sempre seria o mesmo, o mínimo.

3) Busca de Custo Uniforme

A função custo do StayEastSearchAgent aumenta para posições mais à esquerda do mapa e diminui para posições à direita. Assim, o agente prioriza a exploração do lado leste da sala o tanto quanto pode, e apenas vai para o lado oeste quando não há outra opção para atingir o objetivo. De maneira similar, a função custo do StayWestSearchAgent aumenta para posições à direita e diminui para a esquerda, de maneira que o agente imediatamente busca o lado oeste do mapa e vai ao leste apenas quando não tem escolha, retornando à esquerda em direção ao objetivo assim que pode.

4) Busca Heurística

O algoritmo A* tende a ser mais eficiente por possuir conhecimento específico do problema, representado pela função heurística. Por meio dela, o algoritmo é capaz de estimar quais nós têm mais potencial e devem ser explorados primeiro. A razão para se implementar uma heurística consistente é para garantir a otimalidade da solução na versão em grafo. Caso fosse uma busca em árvore, seria necessário apenas que a heurística fosse admissível.

5) Busca Heurística LRTA*

O LRTA* é um algoritmo de busca online. Toma decisões locais e aprende com o ambiente ao longo de múltiplos trials. O número total de nós expandidos cresce acumuladamente (de 3.658 para 18.874), pois o algoritmo continua explorando e refinando seu mapa mental a cada nova tentativa, enquanto algoritmos offline terminam após encontrar uma única solução.

À medida que o número de trials aumenta, o valor estimado do estado inicial tende a se tornar mais preciso. Seus dados mostram uma oscilação inicial no Custo (subindo de 76 para 130 e depois caindo), o que indica que o algoritmo primeiro explorou áreas desconhecidas (aumentando o custo aparente) para depois "aprender" caminhos mais eficientes. O valor da heurística sobe conforme o Pacman descobre que o caminho não é tão curto quanto a estimativa inicial otimista sugeria.

A convergência pode ser verificada pela estabilização do Custo. O algoritmo converge quando o custo da solução para de diminuir e o agente passa a repetir o mesmo caminho ótimo em todos os trials subsequentes.

A heurística nula exige muito mais exploração inicial (mais nós expandidos e custos altíssimos nos primeiros trials) até que o mapa seja desvendado pelo aprendizado. A distância de Manhattan fornece um caminho imediato ao agente. A vantagem é que ela reduz drasticamente o custo e o número de nós nos primeiros trials, pois se aproxima das coordenadas do objetivo, em vez de andar aleatoriamente.

Melhorias

Tive dificuldades com o autograder. Em como melhorar e cobrir tudo. No meu teste ficou 15/18 e não consegui achar onde estavam os problemas.